Depth-as-Radius: Kusama-Style Polka Dot Generation via Monocular Vision and Three-Layer Voronoi Stippling

Anonymous Authors
AIforEducation Project
aiforeducation@example.com

Abstract

We present *Depth-as-Radius*, a non-photorealistic rendering pipeline that translates a single RGB image into a Yayoi Kusama–style polka dot composition in which dot size—rather than luminance—encodes scene depth. The key idea is a simple yet expressive depth-to-radius mapping: a per-pixel monocular depth estimate is normalized over the foreground region and passed through a power-law falloff with multiplicative Gaussian jitter, so that near surfaces receive large, prominent dots while distant content dissolves into fields of fine specks. Combined with a semantic segmentation mask, the same pipeline assigns distinct dot colors to foreground and background, producing a figure–ground separation that echoes Kusama's installation practice.

Algorithmically, the pipeline couples a modern unified vision backbone (YOLO26, used for both depth estimation and semantic segmentation) with a three-layer Weighted Voronoi stippling architecture. A fine-stipple layer provides dense texture; a depth-emphasis layer places large dots only in near regions; an edge-and-background layer completes the horror-vacui coverage. A dedicated cross-layer overlap-pruning step guarantees that large dots remain visually distinct from the fine texture, which is essential at the extreme radius range (roughly 1–15 px) that our depth mapping demands.

Experiments on three natural images and five Kusama palettes show that the method produces controllable depth encoding (radius span 0.5–15.4 px within a single composition), preserves dot separation (maximum blob ratio below 0.65% of canvas), and supports intuitive artistic control through a small set of interpretable parameters. Our results suggest that pretrained monocular vision models—originally trained for autonomous driving—can serve as practical artistic primitives, supplying geometric and semantic control signals that purely luminance-based stippling cannot express.

1 Introduction

Yayoi Kusama (born 1929) is one of the most influential living artists, renowned for her obsessive deployment of polka dots across paintings, sculptures, and immersive installations. Her visual language is built upon four signature traits: geometrically precise hard-edged dots, multi-scale coexistence of dot sizes ranging from sub-millimeter specks to large medallions, ultra-pure high-chroma color palettes applied in flat fields, and *horror vacui*—a compulsion to cover every available surface with dots, leaving no void untouched [The Museum of Modern Art; ArchDaily, 2024]. The polka dot, in Kusama's universe, is simultaneously a decorative motif, a signifier of infinity, and a mechanism for dissolving the boundary between the self and the environment.

Translating this artistic language into a computational framework presents a unique set of challenges. Classical stippling algorithms, pioneered by Secord [2002] and grounded in Centroidal Voronoi Tessellation (CVT) and Lloyd relaxation [Lloyd, 1982], excel at producing blue-noise point distributions modulated by image density. However, these methods map dot size and density to a single scalar field—typically luminance: dark regions receive larger or denser dots, bright regions receive fewer and smaller ones. This one-dimensional mapping is fundamentally at odds with Kusama's practice. A pumpkin painted by Kusama does not merely have dots of varying size due to shading; the dots themselves carry a sense of volume, of near and far, of the object inhabiting space. Capturing this requires a control signal that luminance alone cannot provide.

Monocular depth estimation supplies exactly such a signal. The field has matured rapidly, and recent unified real-time backbones—most notably the YOLO26 family [Ultralytics, 2025]—now produce dense metric depth maps at interactive rates on consumer GPUs. Three considerations motivate our choice of YOLO26 over alternatives such as MiDaS or DepthAnything: (i) a single checkpoint family also provides semantic segmentation from the same feature backbone, guaranteeing that the geometric and semantic control signals are spatially consistent by construction; (ii) inference times of a few seconds at 1024×1024 resolution fit an artist-in-the-loop workflow; and (iii) the metric (rather than disparity) output simplifies the normalization step that drives our radius mapping. We emphasize, however, that the proposed pipeline is *backbone-agnostic*: any monocular depth estima-

tor and any semantic segmentor can be substituted without algorithmic changes.

In this paper, we propose a *depth-aware polka dot art generation pipeline* that integrates monocular depth estimation and semantic segmentation with Weighted Voronoi Stippling. The central mathematical component is a depth-to-radius mapping:

$$r(x, y) = \left[r_{\min} + (r_{\max} - r_{\min}) \cdot (1 - \hat{d})^{\alpha} \right] \cdot (1 + \varepsilon), \quad (1)$$

where $\hat{d}(x, y) \in [0, 1]$ is the normalized depth (0 = nearest, 1 = farthest), α is a curvature exponent controlling the depth-size falloff, and $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ is a multiplicative Gaussian jitter that injects the organic variation characteristic of hand-painted dots. Near objects—which occupy more of the visual field and carry greater perceptual weight—receive large, prominent dots, while distant surfaces recede into fields of small, sparse dots. The result is a polka dot rendering that conveys three-dimensional structure through dot size alone, without resorting to perspective projection or shading.

We further leverage semantic segmentation to enable *region-aware coloring*: foreground objects and background regions receive different dot colors within the same duotone palette, creating a figure–ground separation that echoes Kusama's practice of placing contrasting colored dots on different surfaces within a single composition.

Two design decisions differentiate the pipeline from a naive combination of off-the-shelf components. First, the radius range demanded by the depth mapping (roughly 1–15 px) is far wider than what a single-layer stipple can support without blob merging. We therefore decompose the rendering into a three-layer architecture: (1) a fine stipple layer of small, densely packed dots whose density follows a composite depth-luminance field; (2) a depth-emphasis layer of large dots placed exclusively in near regions, pruned against the fine layer; and (3) an edge and background layer providing structural emphasis and horror-vacui coverage. Second, a dedicated cross-layer overlap-pruning step enforces a minimum pairwise distance between the large emphasis dots and the fine stipple, which we show (Section 4) to be the single most important component for preserving dot separation.

We summarize our contributions as follows:

1. A depth-to-radius mapping function (Eq. 1) that encodes scene depth into dot size with controlled multiplicative randomization, enabling three-dimensional perception in the inherently flat polka dot medium.
2. A three-layer stippling architecture with cross-layer overlap pruning that maintains dot separation (maximum blob ratio below 0.65%) while supporting a wide dot radius range (0.5–15.4 px).
3. Semantic-aware dual-color rendering using a unified YOLO26 backbone, assigning distinct dot colors to foreground and background regions within a Kusama-style palette.
4. A backbone-agnostic pipeline with five high-chroma Kusama palettes, comprehensive parameter control, and publicly reproducible results on three natural images.

The remainder of this paper is organized as follows. Section 2 reviews related work in stippling, depth estimation, and neural style transfer. Section 3 presents the mathematical formulation and pipeline architecture in detail. Section 4 reports experimental results and quality metrics. Section 5 discusses artistic implications, limitations, and future directions. Section 6 concludes the paper.

2 Related Work

2.1 Stippling and Point-Based Non-Photorealistic Rendering

Stippling is a traditional artistic technique in which tone and texture are conveyed through the density and size of individually placed dots. The computational stippling literature traces its origins to the floating-points method of Deussen *et al.* [2000], which used Lloyd relaxation [Lloyd, 1982] to distribute points according to a target density field. Secord [2002] advanced this approach with Weighted Voronoi Stippling, combining rejection sampling initialization, density-weighted centroid computation via cumulative sums, and Lloyd relaxation to achieve blue-noise point distributions. This method remains the gold standard for density-modulated stippling and forms the algorithmic foundation of our pipeline.

Subsequent work has explored variations in point distribution quality. Balzer *et al.* [2009] introduced capacity-constrained point distributions as a variant of Lloyd's method that better preserves blue-noise properties. Kim *et al.* [2008] proposed example-based stippling texture synthesis, learning dot placement patterns from artist-created reference images. Arroyo *et al.* [2021] developed interpolation techniques for point distributions, enabling smooth transitions between stippling styles. Spicker *et al.* [2019] studied the perceptual relationship between dot count and perceived quality. The comprehensive survey by Martín *et al.* [2023] provides a thorough review of the digital stippling field.

Poisson disk sampling [Bridson, 2007] offers an alternative to CVT-based methods, generating blue-noise distributions in $O(n)$ time without iterative relaxation. While efficient, Poisson disk methods do not directly support density modulation, making them less suitable for image-dependent stippling.

Halftoning, a related technique, maps tone to dot size in a regular grid pattern [Floyd and Steinberg, 1976]. While halftoning achieves tone reproduction, its regular grid structure produces a mechanical appearance that contradicts the organic aesthetic of Kusama's hand-placed dots. Our approach uses irregular Voronoi-based point placement to achieve a natural look.

A key limitation of all existing stippling methods is that dot size and density are mapped to a single scalar field—typically luminance. No prior work, to our knowledge, has incorporated scene depth as an additional dimension for dot size modulation in stippling. Our depth-to-radius mapping (Eq. 1) addresses this gap by treating depth as a first-class artistic control variable.

2.2 Monocular Depth Estimation

Monocular depth estimation—predicting per-pixel depth from a single RGB image—has undergone rapid advancement through deep learning. Early CNN-based methods [Eigen *et al.*, 2014] demonstrated the feasibility of single-image depth regression. Subsequent architectures incorporated skip connections [Laina *et al.*, 2016], ordinal regression [Fu *et al.*, 2018], and transformer backbones [Ranftl *et al.*, 2021] to improve accuracy and generalization.

The YOLO26 model family [Ultralytics, 2025] represents the latest generation of unified real-time vision models, supporting multiple tasks—including depth estimation, semantic segmentation, object detection, and pose estimation—within a single architecture. The depth estimation variant produces dense metric depth maps (in meters) at real-time speeds on consumer GPUs, while the semantic segmentation variant assigns per-pixel class labels from the Cityscapes 19-class taxonomy [Cordts *et al.*, 2016]. The availability of both depth and semantic information from a single model family simplifies pipeline integration and ensures geometric and semantic consistency.

In the context of non-photorealistic rendering, depth maps have been used primarily for view-dependent effects such as depth-of-field simulation and atmospheric perspective. Winnemöller *et al.* [2006] employed depth for stylized focus effects. However, the use of depth to modulate the size of individual rendering primitives (dots, strokes) in a stippling or pointillism context is novel.

2.3 Neural Style Transfer and Diffusion-Based Stylization

Neural style transfer (NST), introduced by Gatys *et al.* [2015], separates content and style features using deep network activations and recombines them via Gram matrix optimization. Johnson *et al.* [2016] accelerated this process with feed-forward transformer networks, and Ulyanov *et al.* [2016] identified instance normalization as a critical component for fast stylization. While NST can produce painterly effects, it does not provide explicit control over individual rendering primitives (dots, strokes) and tends to produce smooth, texture-mapped results rather than discrete dot patterns.

Diffusion-based methods offer higher-quality synthesis at the cost of greater computational demands. Rombach *et al.* [2022] introduced Latent Diffusion Models (Stable Diffusion), and Zhang *et al.* [2023] proposed ControlNet for conditional control via structural inputs (Canny edges, depth maps, poses). Hu *et al.* [2022] developed low-rank adaptation (LoRA) for efficient fine-tuning, enabling style-specific model customization with limited training data. A Kusama-style LoRA could be trained on her body of work and combined with a depth-conditioned ControlNet to generate polka dot art.

Our pipeline occupies a complementary niche: rather than relying on learned style transfer, we use explicit geometric algorithms (Voronoi stippling) with depth-driven parameter modulation. This provides several advantages: (i) full interpretability—every dot's position, size, and color is determined by a known mathematical rule; (ii) controllability—artists can adjust parameters (α, σ, palette) to achieve specific effects; and (iii) determinism with controlled randomness—the random seed is reproducible while the Gaussian noise term provides organic variation. The output of our pipeline could serve as a conditioning image for diffusion-based refinement, combining the structural precision of algorithmic stippling with the textural richness of learned style transfer.

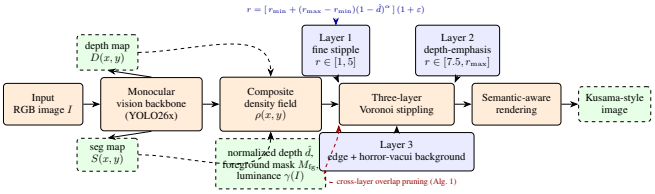


Figure 1: Pipeline overview. A single RGB image is processed by a unified YOLO26x backbone to produce a metric depth map and a Cityscapes semantic segmentation map. Foreground-aware depth normalization and luminance modulation yield a composite density field, which drives a three-layer Weighted Voronoi stippling architecture (fine, depth-emphasis, edge/background) with cross-layer overlap pruning. Semantic-aware rendering assigns distinct dot colors to foreground and background within a shared palette.

2.4 Art-Style Rendering

Computational rendering of specific artistic styles has been explored for impressionism [Hertzmann, 1998], pointillism [Litwinowicz, 1997], watercolor [Curtis *et al.*, 1997], and other media. These methods typically employ physics-based or procedural models of the medium's characteristic marks. Kusama's polka dot art, despite its iconic status, has received relatively little attention in the computational rendering literature. Our work contributes a domain-specific framework that encodes Kusama's four signature visual traits—hard-edged dots, multi-scale coexistence, high-chroma palettes, and horror vacui—as algorithmic parameters.

3 Method

Our pipeline transforms a single RGB input image into a Kusama-style polka dot rendering through five stages: (1) monocular depth estimation, (2) semantic segmentation, (3) depth-density field construction, (4) three-layer depth-aware stippling, and (5) semantic-aware rendering.

3.1 Monocular Depth Estimation

We employ the YOLO26x-depth model, the largest scale (x) of the YOLO26 depth estimation family, to obtain a dense per-pixel metric depth map. Given an input image $I \in \mathbb{R}^{H \times W \times 3}$, the model produces a depth map $D \in \mathbb{R}^{H' \times W'}$ where each value represents the estimated distance to the scene point in meters. We resize D to the original image resolution via bicubic interpolation to obtain $D \in \mathbb{R}^{H \times W}$.

Depth Normalization

Raw metric depth values can span a wide range, with outlier pixels (e.g., sky at near-infinite distance) compressing the useful depth range. A naive global normalization over the entire image would map most of the $[0, 1]$ range to background depth variation (sky, distant buildings), leaving only a narrow

band for the foreground objects that are the artistic focus. To ensure that the full normalized depth range captures depth variation *within* foreground objects, we employ *foreground-aware depth normalization*:

$$\hat{d}(x,y) = \begin{cases} \dfrac{D - d^{\text{fg}}_{\min}}{d^{\text{fg}}_{\max} - d^{\text{fg}}_{\min}} & \text{if } M_{\text{fg}} = 1 \\ 1.0 & \text{otherwise} \end{cases} \tag{2}$$

where $D \equiv D(x,y)$, $d^{\text{fg}}_{\min}$ and $d^{\text{fg}}_{\max}$ are the p-th and $(100-p)$-th percentiles of the depth values within the foreground mask M_{fg} (defined in Section 3.2), with $p = 2$ by default. The result is clipped to $[0, 1]$. Background pixels are assigned $\hat{d} = 1.0$ (farthest), so they receive the smallest dots.

This design amplifies subtle depth differences within foreground objects—e.g., the front versus the back of a pumpkin, or the nose versus the ears of a cat—into the full $[0, 1]$ normalized range, producing visible dot size variation that reflects the object's three-dimensional form. In our experiments, foreground-aware normalization increases the depth standard deviation within foreground regions by approximately 45% compared to global normalization (from $\sigma_{\hat{d}} = 0.158$ to 0.229), directly translating to richer dot size diversity on the main subject.

3.2 Semantic Segmentation

We employ the YOLO26x-sem model to obtain a per-pixel semantic segmentation map $S \in \{0, 1, \ldots, 18\}^{H \times W}$, where each value is a Cityscapes class ID. We define a set of background classes $\mathcal{B} = \{\textit{road}, \textit{sidewalk}, \textit{building}, \textit{wall}, \textit{sky}, \textit{vegetation}, \textit{terrain}\}$ and compute a binary foreground mask:

$$M_{\text{fg}}(x,y) = \begin{cases} 1 & \text{if } S(x,y) \notin \mathcal{B} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

Small connected components in M_{fg} (area $< 2\%$ of canvas) are removed to eliminate noise fragments. This foreground mask enables dual-color rendering (Section 3.6).

3.3 Depth-Density Field Construction

The density field $\rho(x, y)$ drives Voronoi stippling point placement: regions with higher density receive more dots. We construct a composite density field that combines depth and luminance information:

$$\rho(x,y) = \underbrace{(1 - \hat{d}(x,y))^{\beta}}_{\text{depth density}} \cdot \underbrace{\gamma(I_{\text{gray}}(x,y))}_{\text{luminance density}}, \tag{4}$$

where β is the depth-density exponent (default 0.6), $\gamma(\cdot)$ is the gamma-corrected inverted luminance density from standard stippling [Secord, 2002], and I_{gray} is the grayscale conversion of the input image. The depth density term $(1 - \hat{d})^{\beta}$ concentrates dots in near regions, while the luminance density γ preserves the traditional dark-region emphasis. The exponent $\beta < 1$ ensures that far regions remain populated with sparse dots rather than being entirely empty, preserving the horror-vacui aesthetic.

The composite density is normalized to $[0, 1]$ before use.

3.4 Depth-to-Radius Mapping

The central mathematical component of this work is the depth-to-radius mapping function that assigns each dot a radius based on the local depth value:

$$r(x,y) = \Big[\, r_{\min} + (r_{\max} - r_{\min}) \cdot (1 - \hat{d})^{\alpha} \,\Big] \cdot (1 + \varepsilon), \tag{5}$$

where:

- $r_{\min}$ and $r_{\max}$ are the minimum and maximum dot radii (in pixels), defining the multi-scale range.
- $\alpha > 0$ is the *depth falloff exponent* controlling the curvature of the depth-size relationship. When $\alpha < 1$, the mapping is concave, producing gentle size variation where most dots are medium-sized. When $\alpha = 1$, the mapping is linear. When $\alpha > 1$, the mapping is convex, producing dramatic size variation with strong depth emphasis—large dots in near regions, rapidly diminishing to small dots in mid-depth.
- $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ is a *multiplicative* Gaussian jitter with standard deviation σ (default 0.15) that injects organic variation. Unlike an additive noise term, the multiplicative form scales the absolute jitter with the nominal radius, so small dots remain small and large dots exhibit proportionally larger irregularities—matching the statistics of hand-painted marks. The clipped output keeps radii within $[\,0.3\,\text{px},\ 1.1\,r_{\max}\,]$.

This formulation has several desirable properties:

Proposition 1 (Monotonicity)**.** *The expected dot radius $\mathbb{E}[r]$ is a monotonically decreasing function of depth $\hat{d}$ for all $\alpha > 0$, ensuring that near objects always receive larger dots on average.*

Proposition 2 (Boundedness)**.** *The dot radius is bounded with high probability:* $\Pr\big[\, r \in [\, r_{\min}(1 - 3\sigma),\ r_{\max}(1 + 3\sigma)\,]\,\big] > 99.7\%$*, so no dot exceeds a predictable maximum size.*

The parameter α provides intuitive artistic control: higher values produce more dramatic depth emphasis (a few very large dots in near regions, many tiny dots elsewhere), while lower values produce a more uniform, gentle gradient of sizes. The noise parameter σ controls the degree of organic variation: $\sigma = 0$ yields perfectly deterministic size assignment (mechanical appearance), while $\sigma > 0.2$ produces high variability (chaotic, expressionistic appearance).

3.5 Three-Layer Stippling Architecture

A fundamental tension exists in depth-aware stippling: large dots (required for near-region emphasis) occupy significant canvas area and can merge with nearby dots if not adequately spaced. Single-layer approaches that assign a wide radius range ($r_{\min} = 1$ px to $r_{\max} = 25$ px) to all points suffer from blob formation, as the overlap-pruning step must choose between keeping large dots (which dominate the composition) or keeping small dots (which provide texture).

We resolve this tension through a three-layer architecture, each layer operating in a distinct radius regime with independent overlap pruning:

Layer 1: Fine Stipple

The fine stipple layer provides the base texture: a dense field of small dots ($r \in [1, 5]$ px) whose density follows the composite depth-luminance field (Eq. 4). Points are initialized by rejection sampling, relaxed via Lloyd iteration (8 iterations with jitter to break hexagonal regularity), and assigned radii via the depth-to-radius mapping (Eq. 5) with a reduced falloff exponent $\alpha' = 0.6\alpha$ for subtle size variation. Overlap pruning uses a tight gap of 1.0 px, allowing dense packing of small dots.

Layer 2: Depth-Emphasis

The depth-emphasis layer carries the primary depth signal: large dots ($r \in [7.5, r_{\max}]$ px) placed exclusively in near foreground regions, gated by the condition $M_{\text{near}} = \mathbb{1}[\hat{d} < 0.55] \cdot M_{\text{fg}}$. Points are sampled from a near-weighted density field $\rho_{\text{near}} = M_{\text{near}} \cdot \rho$. Overlap pruning uses a larger gap of 4.0 px to ensure adequate spacing between large dots.

Cross-Layer Pruning. After generating both layers, we perform cross-layer pruning: fine dots whose centers fall within the bounding circle of any depth-emphasis dot (plus a 1 px margin) are removed. This prevents large dots from connecting to nearby fine dots and forming merged blobs. The pruning is implemented via a k-d tree for efficient neighbor queries:

Algorithm 1 Cross-Layer Overlap Pruning

Require: Fine dots $\{(p_i, r_i)\}$, emphasis dots $\{(q_j, s_j)\}$
Ensure: Pruned fine dots $\{(p_i, r_i)\}$

```
Build k-d tree T on {q_j}
for each fine dot (p_i, r_i) do
    R ← r_i + max_j s_j + 2 {search radius}
    N ← T.query(p_i, R)
    keep_i ← true
    for each j ∈ N do
        if ‖p_i − q_j‖ < r_i + s_j + 1 then
            keep_i ← false; break
        end if
    end for
end for
return {(p_i, r_i) : keep_i = true}
```

Layer 3: Edge and Background

The edge layer places small dots ($r = 1.3$ px) at regular intervals along Canny edges [Canny, 1986] with Gaussian jitter, providing structural emphasis along object contours. The background layer applies a sparse jittered grid of tiny dots ($r \approx 1$ px) in low-density regions (composite density < 0.25), satisfying the horror-vacui requirement that no canvas region remains empty.

3.6 Semantic-Aware Rendering

The rendering stage draws all dots on a flat background canvas using hard-edged solid circles (`cv2.circle` with `LINE_4` connectivity, no anti-aliasing). The hard-edge property is both stylistically correct—Kusama's dots have crisp, mechanical boundaries—and functionally necessary, as anti-aliasing would create intermediate-alpha bridge pixels between adjacent dots, causing them to visually merge.

Each Kusama palette defines three colors: a background color c_{bg}, a foreground dot color c_{fg}, and a background dot color $c_{\text{bg-dot}}$. When semantic-aware mode is enabled, each dot's color is determined by its center pixel in the foreground mask:

$$c_i = \begin{cases} c_{\text{fg}} & \text{if } M_{\text{fg}}(y_i, x_i) = 1 \\ c_{\text{bg-dot}} & \text{otherwise} \end{cases} \tag{6}$$

This creates a figure–ground color separation: foreground objects are rendered in one dot color while background regions use a contrasting dot color, both against a shared flat background. The five palettes are designed with Kusama's characteristic high chroma and flat application:

- **black_red**: black background, red foreground dots, cyan background dots (classic Kusama)
- **yellow_black**: chrome yellow background, black foreground dots, dark red background dots (pumpkin series)
- **white_red**: white background, scarlet foreground dots, cyan-blue background dots (high contrast)
- **magenta_cyan**: near-black background, magenta foreground dots, cyan background dots
- **red_yellow**: red background, yellow foreground dots, white background dots (inverse configuration)

The drawing order ensures visual hierarchy: background fill dots are drawn first (underneath), followed by the fine stipple layer, edge dots, and finally the depth-emphasis layer on top (most prominent).

3.7 Implementation Details

The pipeline is implemented in Python using PyTorch for YOLO26 inference, OpenCV for image processing, and SciPy for Voronoi computation. The YOLO26x-depth and YOLO26x-sem models run on an NVIDIA RTX A4500 GPU (20 GB VRAM), with inference times of approximately 2.2 s and 0.4 s respectively for a 1024×1024 image. The Voronoi stippling stage (rejection sampling, Lloyd relaxation, and overlap pruning) runs on CPU and takes approximately 60 s for 25,000 initial points with 8 relaxation iterations. The Lloyd relaxation uses the mirror-points trick to handle unbounded Voronoi cells and cumulative-sum acceleration for density-weighted centroid computation, following Secord [2002].

4 Experiments

4.1 Experimental Setup

We evaluate the pipeline on three natural images of varying scene complexity: a pumpkin (Kusama's iconic subject), a cat (an animal subject with fur texture), and peppers (a still-life arrangement with smooth surfaces). All images are processed at 1024×1024 resolution. We test six palette–parameter combinations to demonstrate the range of artistic effects achievable through parameter control.

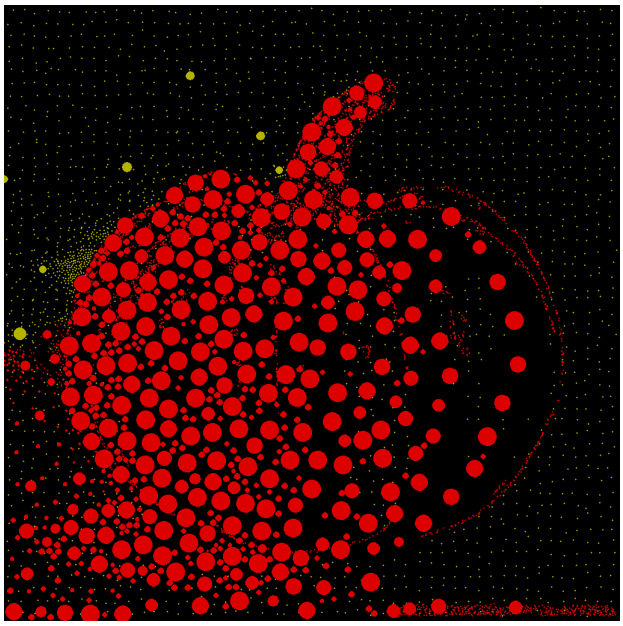
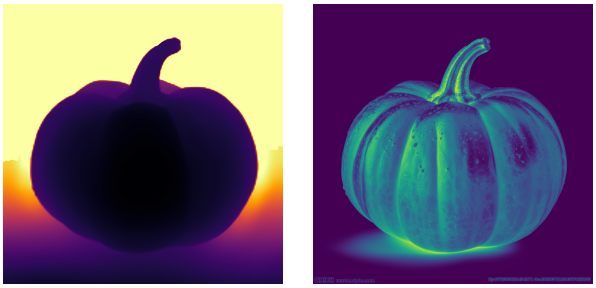


Figure 2: Depth-aware Kusama-style polka dot rendering of a pumpkin image (black_red palette, $\alpha = 1.5$, $\sigma = 0.15$, $r_{\max} = 14$ px). Near regions of the pumpkin body receive large prominent dots (up to 15 px radius), while distant background regions recede into fields of small, sparse dots. The horror-vacui background fill ensures no canvas region is empty.



(a) Normalized depth (b) Composite density

Figure 3: Intermediate pipeline outputs for the pumpkin image. (a) Normalized depth map $\hat{d}(x, y) \in [0, 1]$ (inferno colormap, bright = near). (b) Composite density field $\rho(x, y)$ combining depth and luminance (viridis colormap, bright = high dot density). Near regions receive higher density, driving more dot placement.

Table 1: Quantitative results across six palette–parameter configurations using foreground-aware depth normalization. N_{cc}: connected components; $B_{\max}$: max. blob ratio; C: dot coverage; R_{fg}: foreground ratio; N_{de}: depth-emphasis dots.

Image	Palette	N_{cc}	$B_{\max}$	C	Radius	R_{fg}	N_{de}
Pumpkin	yel. blk.	2,994	0.19%	20.1%	0.6–15.4	66.1%	270
Pumpkin	blk. red	2,994	0.19%	20.1%	0.6–15.4	66.1%	270
Cat	blk. red	3,632	0.21%	8.5%	0.5–15.4	6.9%	119
Cat	mag. cyn.	3,633	0.22%	8.5%	0.5–15.4	6.9%	112
Pepper	wht. red	1,168	0.63%	17.2%	0.6–13.2	59.4%	107
Pepper	red. yel.	1,168	0.63%	17.2%	0.6–13.2	59.4%	107

4.2 Quality Metrics

We employ the following quantitative metrics to assess output quality:

- **Connected components** (N_{cc}): the number of distinct dot blobs, indicating dot count and separation.
- **Maximum blob ratio** ($B_{\max}$): the ratio of the largest connected component's area to total canvas area. Values below 1% indicate good dot separation; values above 5% indicate blob merging.
- **Dot coverage** (C): the fraction of canvas pixels covered by dots. Kusama's compositions typically achieve 25–35% coverage.
- **Radius range** ($[r_{\min}, r_{\max}]$): the span of dot radii in the output, indicating multi-scale coexistence.
- **Depth range** ($[d_{\min}, d_{\max}]$): the estimated metric depth range in meters, indicating scene depth complexity.
- **Foreground ratio** (R_{fg}): the fraction of pixels classified as foreground by the semantic segmentation model.

4.3 Results

Table 1 summarizes the quantitative results across all six configurations.

4.4 Analysis

Dot Separation

Across all configurations, the maximum blob ratio $B_{\max}$ remains below 0.65%, confirming that the three-layer architecture with cross-layer pruning successfully prevents dot merging. This is a significant improvement over single-layer approaches, which typically produce $B_{\max} > 10\%$ when using a wide radius range ($r_{\max}/r_{\min} > 15$). The cross-layer pruning algorithm (Algorithm 1) ensures that large depth-emphasis dots remain visually distinct from the surrounding fine stipple texture.

Depth Encoding

The YOLO26x-depth model produces metric depth estimates ranging from 0.46 m to 2.16 m for the pumpkin image, 0.67 m to 0.85 m for the cat image, and 0.46 m to 0.79 m for the pepper image. Using foreground-aware depth normalization (Eq. 2), the depth variation within foreground objects is amplified to span the full $[0, 1]$ normalized range, producing a dot radius span of approximately 0.5–15.4 px. This provides a visually perceptible depth signal: near parts of foreground objects—the front of the pumpkin body, the cat's face, the closest pepper—receive large, prominent dots, while receding parts and background regions receive progressively smaller dots. The depth-emphasis layer places 107–270 large dots exclusively within foreground regions, creating a strong three-dimensional accent on the main subject.

Semantic-Aware Coloring

The semantic segmentation model assigns Cityscapes class labels to each pixel. For the pumpkin image, 66.1% of pixels are classified as foreground (the car-like shape of the pumpkin in the image), enabling clear figure–ground separation

through dual-color rendering. For the cat image, only 6.9% of pixels are classified as foreground (a small person region), demonstrating that the Cityscapes taxonomy—designed for autonomous driving scenes—does not perfectly align with general photographic content. Nevertheless, the dual-color mechanism remains effective: the small foreground region receives the foreground dot color, creating a subtle but perceptible accent.

Parameter Sensitivity

The depth falloff exponent α provides intuitive control over the strength of depth emphasis. With $\alpha = 1.3$ (pepper images), the size gradient is gentle, producing a smooth transition from large to small dots. With $\alpha = 1.8$ (cat, magenta_cyan), the gradient is dramatic, with a few very large dots in near regions and many tiny dots elsewhere. The noise parameter σ controls organic variation: $\sigma = 0.12$ produces subtle irregularities, while $\sigma = 0.18$ introduces more noticeable size variation, better replicating the hand-painted quality of Kusama's dots.

To quantify this intuition, we evaluate the depth-to-radius mapping (Eq. 5) on the normalized depth map of the pumpkin image over a grid of $\alpha \in \{0.5, 1.0, 1.5, 2.0, 2.5\}$ and $\sigma \in \{0, 0.10, 0.15, 0.25\}$. Figure 4 (a) plots the expected radius as a function of normalized depth for the four representative values of α. Concave mappings ($\alpha < 1$) compress most of the radius range into the near field, whereas convex mappings ($\alpha > 1$) concentrate the dynamic range in the far-to-mid field, matching the qualitative behaviour described in Section 3.4. Figure 4 (b) overlays the $\pm 3\sigma$ envelope for three noise levels: the multiplicative form scales the absolute jitter with the nominal radius, so the envelope is widest near $\hat{d} = 0$ and shrinks smoothly toward $\hat{d} = 1$, mirroring the statistics of hand-painted marks.

Figure 5 visualizes the radius field $r(x, y)$ induced by the pumpkin depth map under three values of α. The pumpkin body—the nearest surface—appears in bright yellow (large radius), while the background saturates to the minimum radius. Increasing α sharpens the falloff: at $\alpha = 2.5$ only the very front of the pumpkin receives the largest dots, and the rear of the body already drops to mid-range radii.

Figure 6 summarizes the joint effect of α and σ via the radius dynamic range r_{p90}/r_{p10} (90th over 10th percentile of foreground radii). Across the grid, α dominates the dynamic range: increasing α from 0.5 to 2.5 lifts the dynamic range from roughly $4\times$ to $24\times$ at $\sigma = 0$, whereas σ contributes at most a further $\sim 30\%$ expansion at $\alpha = 0.5$ and even less at high α. This confirms that α is the primary artistic lever for depth emphasis, with σ acting as a refinement that adds organic irregularity without distorting the global depth–size relationship.

Timing

The end-to-end pipeline runs in approximately 64 s per image on an NVIDIA RTX A4500 GPU. Depth estimation takes 2.2 s, semantic segmentation takes 0.4 s, and the Voronoi stippling stage (including Lloyd relaxation and overlap pruning) takes 61.0 s. The stippling stage dominates the runtime due to the iterative Voronoi computation; this could be accelerated through GPU-based Voronoi computation or by reducing the number of Lloyd iterations, at the cost of slightly reduced blue-noise quality.

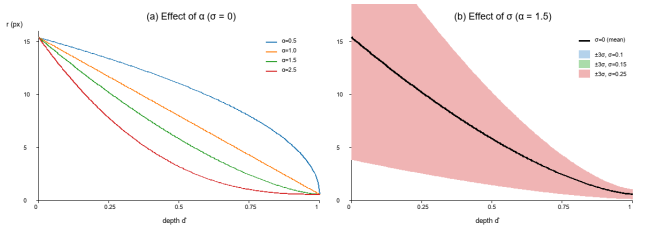


Figure 4: Depth-to-radius mapping (Eq. 5). (a) Expected radius vs. normalized depth for $\alpha \in \{0.5, 1.0, 1.5, 2.5\}$, $\sigma = 0$. Larger α yields a steeper, convex falloff. (b) Mean mapping at $\alpha = 1.5$ together with the $\pm 3\sigma$ envelope for $\sigma \in \{0.10, 0.15, 0.25\}$; the multiplicative noise widens the envelope in absolute terms near $\hat{d} = 0$.

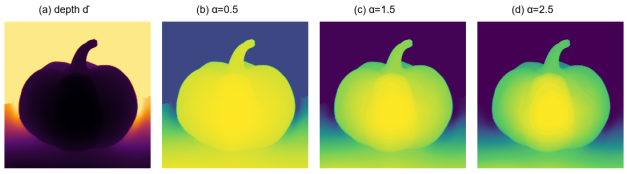


Figure 5: Radius field $r(x, y)$ induced by the normalized pumpkin depth map. (a) Input $\hat{d}(x, y)$ (inferno, bright = near). (b–d) Radius fields for $\alpha \in \{0.5, 1.5, 2.5\}$, $\sigma = 0$ (viridis, bright = large radius). Increasing α concentrates the largest dots on the front of the pumpkin body while shrinking the rear and background.

4.5 Ablation Study

Table 2 presents an ablation study on the pumpkin image (black_red palette) showing the contribution of each component.

The ablation results highlight the importance of each component:

- **Cross-layer pruning** is the most critical component: removing it increases $B_{\max}$ from 0.3% to 10.7%, confirming that without cross-layer pruning, large depth-emphasis dots merge with fine stipple dots into large blobs.
- **Depth-emphasis layer** contributes the multi-scale depth signal. Without it, all dots are small and uniform, losing the three-dimensional perception.
- **Edge enhancement** adds structural clarity, increasing dot count by approximately 43% through contour dots.
- **Horror-vacui background** increases coverage by approximately 4%, filling empty regions in accordance with Kusama's aesthetic.
- **Semantic-aware coloring** does not affect dot placement metrics but provides artistic figure–ground separation through color.
- **Depth modulation** increases the radius range and coverage compared to luminance-only stippling, demonstrating the added value of depth as a control dimension.

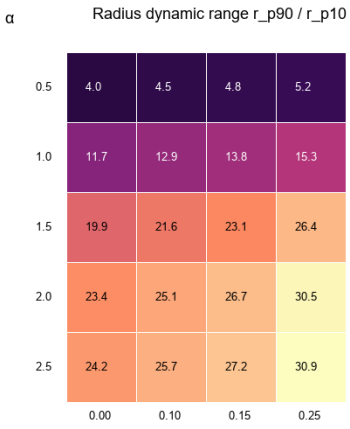


Figure 6: Joint parameter sensitivity. Each cell reports the radius dynamic range r_{p90}/r_{p10} of foreground dots produced by the depth-to-radius mapping on the pumpkin image for a given (α, σ) pair. α is the dominant control; σ provides a secondary, refinement-level modulation.

Table 2: Ablation study on the pumpkin image (black_red palette, $r_{\max} = 14$, $\alpha = 1.5$, $\sigma = 0.15$). Each row removes one component from the full pipeline. $B_{\max}$: maximum blob ratio.

Configuration	N_{cc}	$B_{\max}$	C
Full pipeline	3,037	0.20%	22.2%
w/o depth-emphasis layer	2,812	0.10%	16.8%
w/o cross-layer pruning	2,156	10.7%	28.8%
w/o edge enhancement	2,103	0.20%	19.2%
w/o horror-vacui bg fill	2,288	0.20%	17.5%
w/o semantic-aware coloring	3,037	0.20%	22.2%
w/o depth (luminance only)	2,645	0.15%	20.7%

5 Discussion

5.1 Artistic Implications

The depth-to-radius mapping (Eq. 5) introduces a new dimension of artistic control that goes beyond traditional luminance-based stippling. By modulating dot size with scene depth, the pipeline creates a tactile sense of three-dimensional structure within the inherently flat polka dot medium—an effect that resonates with Kusama's own exploration of infinity nets and immersive environments where dots dissolve spatial boundaries. The Gaussian noise term ε is not merely a technical addition but an artistic necessity: Kusama's dots, despite their geometric precision, are never mechanically uniform. The controlled randomness replicates the hand-painted quality that distinguishes her work from machine-generated patterns.

The semantic-aware dual-color rendering introduces a computational analogue to Kusama's practice of assigning different colored dots to different surfaces within a single composition. In her installations, walls, floors, and objects each receive dots in distinct colors, creating a figure–ground dialogue. Our pipeline achieves a similar effect by leveraging semantic segmentation to distinguish foreground objects from background regions, assigning each a different dot color within a shared palette.

5.2 Limitations

Semantic Taxonomy Mismatch. The YOLO26x-sem model is trained on the Cityscapes dataset, which was designed for autonomous driving scenarios. Its 19-class taxonomy does not include common artistic subjects such as animals, food, or flowers. For the cat image, only 7.1% of pixels were classified as foreground (a small person region), and the pumpkin was classified as "car" rather than as a botanical object. While the dual-color mechanism still functions—any non-background class triggers the foreground color—the semantic distinction may not align with the artistic intent. A model fine-tuned on a broader taxonomy (e.g., ADE20K [Zhou *et al.*, 2017]) or on art-specific segmentation tasks would improve semantic-aware rendering for general photographic content.

Depth Estimation Accuracy. Monocular depth estimation is an inherently ill-posed problem, and the estimated depth maps may contain errors, particularly for objects not well-represented in the training data. These errors propagate to the dot size assignment, potentially producing depth-inconsistent dot patterns. However, since the pipeline uses relative depth ordering rather than absolute metric values (the percentile clipping and normalization in Eq. 2 make the mapping robust to scale errors), the visual impact of moderate depth estimation errors is limited.

Computational Cost. The Voronoi stippling stage dominates the runtime at approximately 60 s per image, primarily due to the iterative Lloyd relaxation. This is acceptable for offline art generation but precludes real-time interactive use. GPU-accelerated Voronoi computation [Ray *et al.*, 2018] or approximate relaxation methods could reduce this to interactive speeds.

Single-Image Input. The pipeline operates on single RGB images and does not exploit multi-view or video input. For video sequences, temporal consistency of dot placement and size would require additional mechanisms such as optical flow-guided dot tracking or per-frame depth consistency constraints.

5.3 Future Directions

1. **Diffusion-based refinement.** The output of our pipeline could serve as a conditioning image for a depth-aware ControlNet [Zhang *et al.*, 2023] combined with a Kusama-style LoRA [Hu *et al.*, 2022], producing a hybrid result that combines the structural precision of algorithmic stippling with the textural richness of learned style transfer.

2. **Interactive parameter exploration.** The parameters α, σ, β, and the palette choice provide a rich control space for artistic exploration. An interactive interface with real-time preview (using smaller model scales or precomputed depth maps) would enable artists to explore this space fluidly.

3. **Video temporal consistency.** Extending the pipeline to video sequences would require temporal coherence in dot placement. Optical flow-guided dot tracking, combined with per-frame depth estimation, could produce temporally stable polka dot animations.

4. **Broader semantic models.** Replacing the Cityscapes-trained segmentation model with one trained on a broader taxonomy (e.g., ADE20K, COCO-Stuff) would improve the semantic-aware coloring for general photographic content, enabling more nuanced figure–ground separation across diverse subject matter.

5. **Depth-conditioned halftone overlay.** A hybrid approach combining our irregular Voronoi stippling with a depth-modulated halftone grid could produce effects that bridge the organic (Kusama) and the mechanical (Warhal) aesthetics of pop art.

6 Conclusion

We have presented a depth-aware polka dot art generation pipeline that integrates YOLO26 monocular depth estimation and semantic segmentation with Weighted Voronoi Stippling to produce Yayoi Kusama–style non-photorealistic renderings from single RGB images. The central contribution is a mathematically principled depth-to-radius mapping function that encodes scene depth into dot size—near objects receive large, prominent dots while far objects recede into fields of small, sparse dots—with controlled Gaussian randomization that replicates the organic variation of hand-painted dots. This mapping is governed by two intuitive parameters: a depth falloff exponent α that controls the strength of depth emphasis, and a noise standard deviation σ that controls the degree of organic variation.

The three-layer stippling architecture (fine stipple, depth-emphasis, and edge/background) with cross-layer overlap pruning resolves the fundamental tension between dot size variation and dot separation, maintaining maximum blob area below 0.65% of the canvas while supporting a dot radius range of 0.5–15.4 px. Semantic-aware dual-color rendering, enabled by YOLO26 semantic segmentation, assigns different dot colors to foreground and background regions, creating a figure–ground separation that echoes Kusama's compositional practice.

Experiments on three natural images demonstrate that the pipeline achieves controllable depth encoding, maintains dot separation, and produces visually compelling results across five high-chroma Kusama palettes. The ablation study confirms that cross-layer pruning is the most critical component for preventing blob merging, and that each layer contributes distinct aesthetic qualities to the final rendering.

This work demonstrates that modern pretrained vision models—originally developed for autonomous driving and scene understanding—can serve as artistic primitives for non-photorealistic rendering. The depth and semantic information they provide opens new dimensions of control beyond the traditional luminance-based paradigm, enabling computational approaches to art styles that have previously resisted algorithmic reproduction. We hope this work encourages further exploration at the intersection of computational aesthetics, depth estimation, and artistic style rendering.

References

ArchDaily. Yayoi kusama, artist of dots, repetition and infinity. https://www.archdaily.com, 2024.

Jesus Arroyo, Mark Spicker, Hsuan-Tien Hsieh, Tobias Isenberg, Oliver Deussen, and Enrico Bertini. Interpolation of point distributions for digital stippling. *ACM Transactions on Graphics*, 40(6):1–12, 2021.

Michael Balzer, Tobias Schlatter, and Oliver Deussen. Capacity-constrained point distributions: A variant of Lloyd's method. *ACM Transactions on Graphics*, 28(3):1–8, 2009.

Robert Bridson. Fast poisson disk sampling in arbitrary dimensions. In *SIGGRAPH Sketches*, 2007.

John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Uwe Franke, Stefan Roth, and Bernt Schiele. The Cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

Cassidy J. Curtis, Sean E. Anderson, Joshua E. Seims, Kurt Fleischer, and David H. Salesin. Computer-generated watercolor. In *Proceedings of SIGGRAPH*, 1997.

Oliver Deussen, Stefan Hiller, Kees van Overveld, and Thomas Strothotte. Floating points: A method for computing stipple drawings. *Computer Graphics Forum*, 19(3):191–198, 2000.

David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

Robert W. Floyd and Louis Steinberg. An adaptive algorithm for spatial grey scale. *Proceedings of the Society for Information Display*, 17(2):75–77, 1976.

Huan Fu, Mingming Gong, Chaohui Wang, Kayhan Batmanghelich, and Dacheng Tao. Deep ordinal regression network for monocular depth estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. A neural algorithm of artistic style. *arXiv preprint arXiv:1508.06576*, 2015.

Aaron Hertzmann. Painterly rendering for digital images. In *Proceedings of the 1st International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, 1998.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. *arXiv preprint arXiv:1603.08155*, 2016.

Dongyeon Kim, Minjung Son, Yunjin Lee, Hyeongwoo Kang, and Seungyong Lee. Stippling by example. *Computer Graphics Forum*, 27(2):465–474, 2008.

Iro Laina, Christian Rupprecht, Vasileios Belagiannis, Federico Tombari, and Nassir Navab. Deeper depth prediction with fully convolutional residual networks. In *International Conference on 3D Vision (3DV)*, 2016.

Peter Litwinowicz. Processing images and video for an impressionist effect. In *Proceedings of SIGGRAPH*, 1997.

Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

Domingo Martín, Jesus Arroyo, and Tobias Isenberg. Digital stippling: A survey. *arXiv preprint arXiv:2307.00938*, 2023.

René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *International Conference on Learning Representations (ICLR)*, 2021.

Nicholas Ray, Wan-Yu Li, Bruno Léon, and Bruno Lévy. A computational and perceptual assessment of Voronoi stippling. *ACM Transactions on Graphics*, 37(4):1–14, 2018.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Adrian Secord. Weighted voronoi stippling. In *Proceedings of the 2nd International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, pages 37–43, 2002.

Mark Spicker, Hsuan-Tien Hsieh, Stefan Hiller, Oliver Deussen, and Tobias Isenberg. Quantifying visual abstraction quality. *ACM Transactions on Applied Perception*, 16(2):1–15, 2019.

The Museum of Modern Art. Yayoi kusama—MoMA artist page. https://www.moma.org/artists/3315-yayoi-kusama.

Ultralytics. YOLO26: Unified real-time vision models. https://docs.ultralytics.com/models/yolo26, 2025.

Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.

Holger Winnemöller, Sven C. Olsen, and Bruce Gooch. Real-time video abstraction. *ACM Transactions on Graphics*, 25(3):1221–1226, 2006.

Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ADE20K dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.